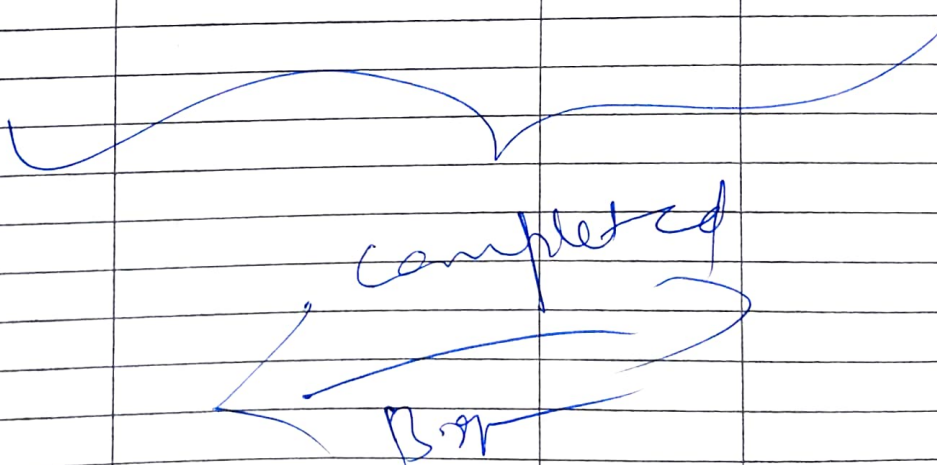
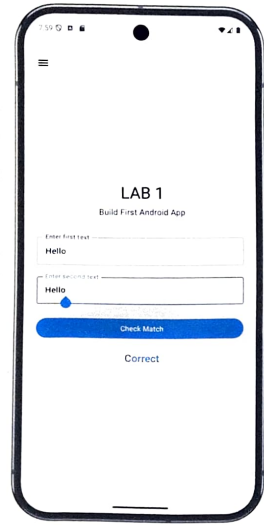


INDEX

NAME: Mansharam K STD.: CSE SEC.: _____ ROLL NO.: 177 SUB.: MAD OBS NOTE

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.		String Comparison	82	
2.		GUI Layout	82	
3.		App Navigation	82	
4.		Activity & Fragment Lifecycle	82	
5.		App Architecture - UI Layer	82	
6.		App Architecture - Persistence Method	82	
7.		Advanced RecyclerView Use Cases	82	
8.		Internet connectivity Detection	82	
9.		Repository Pattern	82	
10.		App UI Design Principles	82	
 Completed B. 91				



Ex: 1

String Comparison

Aim:

To check whether the first text and second text are the same by considering uppercase and lowercase characters.

Algorithm:

Step-1: Start the program.

Step-2: Input the first text string from the user.

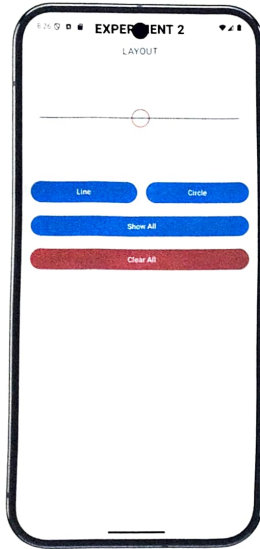
Step-3: Input the second text string from the user.

Step-4: Compare both strings character by character including case sensitivity.

Step-5: If both strings character are identical, print "Correct", otherwise print "Incorrect".

Result:

Thus, the program successfully checks whether two text strings are the same considering uppercase and lowercase characters.



Ex: 2

GUI Layout

Aim:

To design a graphical user interface with buttons to draw a line and a circle, and to visualize them using Show All and Clear All buttons.

Algorithm:

Step-1: Start the program and create a GUI window with a layout.

Step-2: Add buttons for Line, Circle, Show all and Clear all

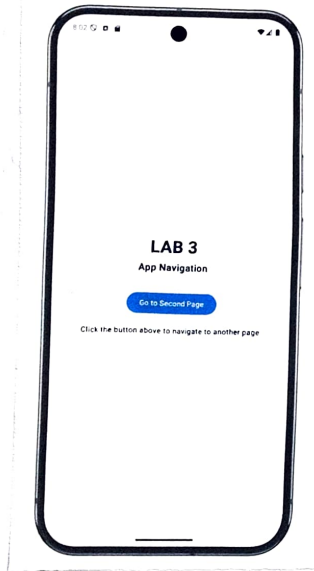
Step-3: When the Line button is pressed, generate the Line

Step-4: When the Circle button is pressed, generate the Circle

Step-5: Use Show all to display shapes and Clear all to remove all shapes from the screen.

Result:

Thus, the GUI application successfully creates and visualizes line and circle shapes using buttons.



Ex: 3

App Navigation

Aim:

To implement navigation between the main page and the second page using buttons.

Algorithm:

Step-1: Create the main page interface.

Step-2: Add a button "Go to second page" on main page.

Step-3: When the button is clicked, navigate to the second page.

Step-4: On the second page, add a button "Go to Main Page".

Step-5: When the button is clicked, navigate back to the main page.

Result:

Thus, the application successfully performs navigation between the main page and the second page using buttons.

EX: 4

Activity 4 Fragment Lifecycle

Aim:

To demonstrate activity and fragment lifecycle events and implement a simple timer.

Algorithm:

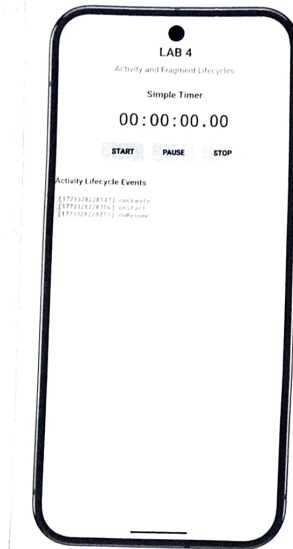
Step-1: Start the application and create an activity with a fragment and a timer display.

Step-2: Display lifecycle events such as `onCreate()`, `onStart()` and `onResume()` when activity starts.

Step-3: Add a start button to begin the timer counting.

Step-4: Add a pause button to temporarily stop the timer.

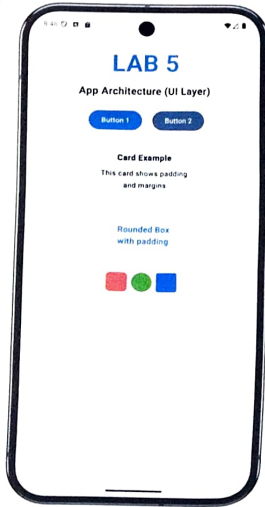
Step-5: Add a stop button to reset the timer to 00:00:00.00.



Result:

Thus, the program successfully demonstrates activity and fragment lifecycle events and implements a simple timer with start, pause and stop functions.

✓



Aim:

To design a simple UI layer using buttons and a card layout with a rounded box, padding and different colors.

Algorithm:

Step-1: Start the application and create the UI layout.

Step-2: Add button1 and button2 to the interface.

Step-3: Create a card component with a rounded box and apply padding.

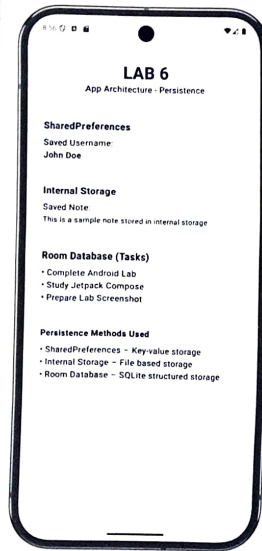
Step-4: Apply different background colors such as red, green, blue to UI elements.

Step-5: Display the UI layer with designed buttons and colored card.

Result:

Thus, the program successfully demonstrates the UI layer of app architecture using button and a colored card layout.

✓



Aim:

To demonstrate data persistence in an application using SharedPreferences, internal storage and room database.

Algorithm:

Step-1: Create the basic app architecture.

Step-2: Use sharedPreferences to store small key value data.

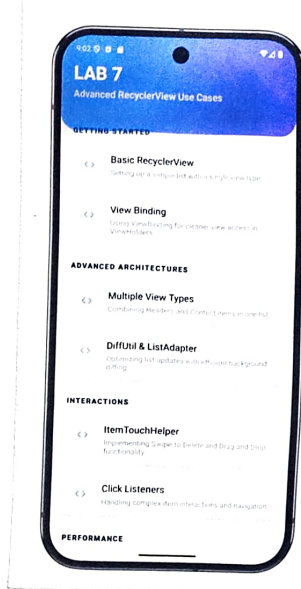
Step-3: Use internal storage to save and retrieve the data.

Step-4: Implement room database to store and manage tasks.

Step-5: Retrieve and display the stored data.

Result:

Thus, the application successfully demonstrates data persistence using shared preferences, internal storage and room database.



Aim:

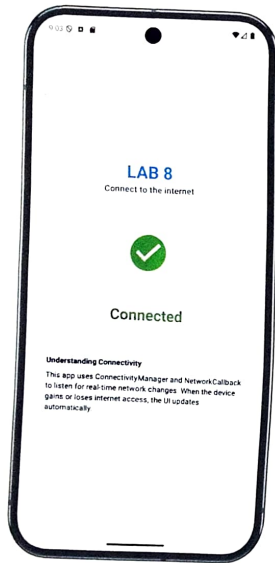
To implement advanced recyclerview features including multiple view types, interactions and performance optimisation.

Algorithm:

- Step-1: Start the application and create a basic RecyclerView.
- Step-2: Implement multiple view types and manage data with DiffUtil and ListAdapter.
- Step-3: Add user interactions using ItemTouchHelper and CheckListeners.
- Step-4: Improve performance by optimizing data updates and view recycling.
- Step-5: Display the list items efficiently using the advanced RecyclerView architecture.

Result:

Thus, the program successfully demonstrates advanced RecyclerView features with interactions and performance optimisation.



Ex: 8

Internet Connectivity Detection

Aim:

To detect internet connectivity and update the UI when the network status changes.

Algorithm:

Step-1: Start the application and check internet connection.

Step-2: Use Connectivity Manager to monitor the network status.

Step-3: Implement NetworkCallback to listen for real time network changes.

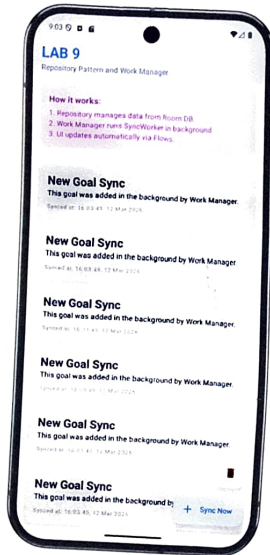
Step-4: Detect when the device gains or loses internet access.

Step-5: Update the UI automatically based on connectivity.

Result:

Thus, the application successfully detects internet connectivity.

✓



Aim:

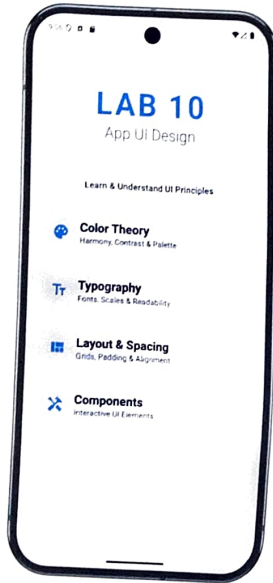
To implement the repository pattern and use WorkManager to perform background synchronization of data.

Algorithm:

- Step-1: Start the application and implement the repository pattern to manage data operations.
- Step-2: Use WorkManager to schedule background tasks.
- Step-3: Create a syncWorker that runs in the background to add or sync new goals.
- Step-4: Execute the worker at specific time and date to perform synchronization.
- Step-5: Update the UI automatically using flows when new data is synced.

Result:

Thus, the application successfully performs background data synchronization using WorkManager and Repository Pattern.



Aim:

To understand and apply basic UI design principles in an application.

Algorithm:

Step-1: Start the application and design the basic user interface.

Step-2: Apply color theory to choose suitable and consistent colors.

Step-3: Use proper typography for readable and attractive text.

Step-4: Maintain proper layout and spacing for better UI structure.

Step-5: Add UI components such as buttons, cards and text elements.

Result:

Thus, the application successfully demonstrates basic UI design principles including color, typography, layout and components.